PolyVote: Making Silent Database Tampering Detectable in Small Ranked-Choice Elections

Tharakesharasu Thennarasu, M Pavan Durga Karthik and Shivam Malge
Vellore Institute of Technology, School of Computer Science and Engineering
Faculty guide: Dr. Katari Balakrishna

Abstract—Student bodies, clubs and departments run elections on spreadsheets, form tools or small web applications in which one administrator can edit the database without leaving a trace. End-to-end verifiable systems [1, 2] remove that trust but need key ceremonies and trustees that small organisations will not operate. We present PolyVote, an open-source instant-runoff voting platform that makes silent edits detectable using ordinary database features: rules enforced inside the relational engine, an HMAC-keyed hash-chained receipt ledger in a separate store, a keyed seal over the voter roster, and an integrity sweep that reconciles ballots, receipts, voter flags and rosters and names the tampered records. We attack the system with 31 storage-level scenarios, run once against a keyed and once against a plain chain, and record which detector fires; ablations and comparisons with other designs are then read from that single record. Against an attacker with write access to the stores but not the ledger key, the final design detects 28 of 28 in-scope scenarios, against 11 for a plain chain with a receipt sweep, 11 for a conventional application whose officer compares counters, and 23 for our previous design. Over 399 sweeps of randomised clean elections there were no false alarms, and the tally agreed with an independent exhaustive implementation in 500 of 500 random elections. The system is not end-to-end verifiable, assumes an honest key holder and has not been evaluated with users; we state these limits.

Index Terms—electronic voting, instant-runoff voting, tamper-evident logging, HMAC, hash chain, database integrity, audit.

I. INTRODUCTION

Most elections are small: a student council, a society, a class representative. They are run on whatever is at hand, usually a single database that its administrator can change. Research on electronic voting has concentrated on protocols that avoid trusting the server [1, 2, 3, 4, 5] and on the failures of deployed internet voting [6, 7, 8, 9]. Between these lies a large population of small elections for which a cryptographic system is impractical, yet where a good deal can be achieved with the database itself.

We ask a narrow question: *what is the cheapest design that makes silent edits by someone with database access detectable?* Our contributions are: (1) a three-store design in which ballots, receipts and voter contact data are held apart, with election rules enforced by the engine; (2) an integrity sweep with six reconciliation rules and a keyed voter-roster seal, which closed a ghost-voter gap that we found in an earlier version; (3) a detector-level evaluation method in which each attack is run once and ablations and design comparisons are read from the same record; and (4) measurements of detection, false alarms, tally correctness, password cost, scaling to 100,000 ballots and concurrency, with the attack scripts, raw data and a documentation survey of 33 related systems released with the code.

II. RELATED WORK

Helios [1] and Belenios [2] give a verifiable tally with encrypted ballots; Civitas [3] adds coercion resistance; Prêt à Voter [4] and Scantegrity II [5] target paper-based verification. Analyses of deployed systems found serious weaknesses in Estonian internet voting [6] and in mobile and web products [7, 8], and a National Academies study advised against internet voting for now [9]; blockchain voting adds further problems [10]. PolyVote does not compete with these on assurance; it targets the settings in which they are not used.

Hash chaining for time-stamping [11] and for secure audit logs [12] underlies tamper-evident logging [13, 14]. In databases, work covers tamper detection in audit logs [15], forensic analysis of tampering [16] and ledger tables in a commercial engine [17]. Instant-runoff outcomes can hinge on few ballots, which motivates auditing [18, 19]. Our difference is the combination: the chain is keyed, it is reconciled against a relational store and a roster seal, and the whole is evaluated by attack.

III. SYSTEM DESIGN

PolyVote is one FastAPI service with a React front end and three stores, separated by what each is trusted with (Fig. 1). The **election database** (SQLite) holds organisations, elections, candidates, the roster and ranked ballot rows. The **receipt ledger** (MongoDB Atlas or a local file) holds one sealed block per ballot. A **verification database** holds voter email addresses, hashed one-time codes and a mail log. The secrets (ledger key, SMTP credentials) live outside all three.

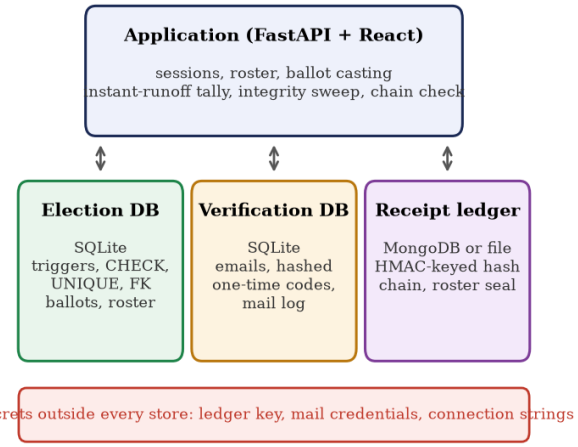


Fig. 1. Architecture: three stores, with secrets held outside all of them.

A. Rules enforced by the engine

A trigger aborts a second update of the voted flag; CHECK constraints bound flags and ranks; a composite key prevents ranking a candidate twice; registration numbers are unique per election; foreign keys tie rows together [20]. A ballot is stored

under a random token and no column links it to a voter. Casting is serialised: the server re-checks the election state, sets the flag and inserts the ranked rows in one transaction, then appends the receipt. A failure between the two leaves a ballot with no receipt, which the sweep reports; we chose the variant that fails visibly. Counting is instant-runoff.

B. Keyed receipt ledger

Each ballot yields a block holding its index, election, token, a hash of the ranked choices, a hash of the candidate list, a roster hash, the previous hash and its own hash. The block hash is HMAC-SHA-256 under a key held outside both stores, so an attacker who can write the ledger but lacks the key cannot rebuild a consistent chain, which a plain hash chain permits. A unique index on the block position stops two writers from forking the chain.

C. Integrity sweep and roster seal

The sweep applies six rules: (1) a ballot whose recomputed hash differs from its receipt is *modified*; (2) a receipt with no ballot is *deleted*; (3) a ballot with no receipt is *injected*; (4) candidate names that differ from the hash sealed in every block are *renamed*; (5) the three counters (voters marked as voted, ballots, receipts) must agree; and (6) the voters in the database must match a keyed seal of the roster, one HMAC tag per voter and a signature over the set, written by the application and kept in the ledger store. Rule 6 detects voters added, deleted or edited outside the application, and the blocks record whether the election was sealed, so deleting the seal is itself detected. A separate chain check recomputes every block hash and link.

D. Authentication

Passwords are stored as PBKDF2-HMAC-SHA256 with a random 16-byte salt and 600,000 rounds [21, 22]; older unsalted hashes verify once and are upgraded at the next sign-in. After the password, a six-digit code is emailed, stored only as a hash, valid for five minutes, single use, limited to three guesses and rate-limited per voter [23]. Email is only as strong as the mailbox [24].

IV. THREAT MODEL

The attacker has write access to the election database, the ledger store, or both, and does **not** hold the ledger key: a rogue database administrator, a ledger-store administrator, or an insider with both stores but no secrets. Three of the 31 scenarios are deliberately outside this model and are reported anyway: two in which the attacker holds the key, and one that edits a candidate's platform text, which is not sealed. We do not claim protection against a malicious key holder, a compromised application host, coercion, vote selling, denial of service or malware on a voter's device.

V. EVALUATION METHOD

Scenarios. We wrote 31 attacks (C1 to C31) at the storage level: swapping or replacing choices, deleting, injecting or stuffing ballots, editing, deleting, truncating, wiping, replaying and reordering ledger blocks, rebuilding a whole chain, renaming candidates, resetting voted flags, adding, deleting or renaming voters, forging or deleting the roster seal, and rolling a store back to an older backup. Each runs against a throwaway copy of the application, so neither Atlas nor real data is touched.

TABLE I. The 31 attack scenarios by family, and what catches them in the final design.

Attack family	IDs	Caught by
Edit a ballot	C1, C2	rule 1
Delete or inject a ballot	C3 to C5	rules 2, 3; counters
Forge receipt, rebuild chain, erase a vote	C6, C10, C27	chain check (needs key)
Tamper with ledger blocks	C7 to C9, C18 to C22	chain check; rules 2, 3
Roll a store back to a backup	C30, C31	rules 2, 3; counters
Reset or fake voted flags	C12, C26	rule 5
Change the candidate list	C11, C24, C25	rule 4
Ghost, deleted or renamed voters; roster seal	C13 to C17	rule 6
Outside the model (key holder; platform text)	C23, C28, C29	not detected

Detector-level record. Every scenario is run once with the keyed chain and once with a plain SHA-256 chain, and we record which detectors fire (chain check, rules 1 to 6, and an officer comparing counters by eye). A *design profile* is a subset of detectors plus a key mode, so its detection is read from the record, not re-implemented. To validate the idea, we emulated our earlier plain-chain design this way and compared it with the real earlier code on C1 to C13: the two agreed on 13 of 13.

Other measurements. Randomised clean elections for false alarms; sweeps after tampering 1, 5 and 20 ballots per category for precision and recall; 500 random elections for tally correctness against an independent implementation; timing from 100 to 100,000 ballots; one to 16 simultaneous voters; password-guessing cost; and probes for abuse of the API and voter privacy. Competing systems were **not** run: we read the documentation of 33 of them and place their documented mechanisms in the nearest measured design class.

VI. RESULTS

A. Detection

TABLE II. Attack scenarios detected by each design.

Design (detectors, key)	All 31	In scope (28)
CRUD application, officer compares counters	11	11
Plain chain only	6	6
Plain chain + receipt sweep (our Review 2 design)	11	11
Plain chain + all six rules	24	24
Keyed chain only	9	9
Keyed chain + rules 1 to 5 (previous design)	23	23
Keyed chain + six rules (PolyVote)	**28**	**28**

The final design detects 28 of 31 scenarios, which is all 28 that lie within the threat model (Table II, Fig. 2). No single control is enough: the strongest detector alone finds 10. The key matters: with a plain chain the same six rules would miss four in-scope attacks, those that forge or rebuild a chain that anyone can recompute. Append-only triggers on the ballot table prevent 8 of 31 against a writer who cannot drop them,

but they do not stop injection, ledger tampering or roster changes, and a writer with schema rights removes them.

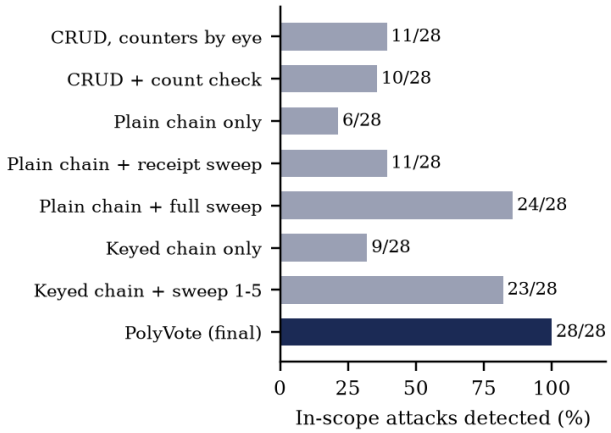


Fig. 2. Share of in-scope attack scenarios detected by each design.

B. Ablation

Removing the chain check loses 7 attacks, the voter seal 5, the candidate seal 3, the count rule 2 and the modified-ballot rule 2 (Fig. 3). Removing rule 2 or rule 3 loses none, because the chain and counters also see those attacks; they remain because they name the records. The roster seal is what closes the ghost-voter attack: the previous design caught 23 of 31 and missed every roster attack.

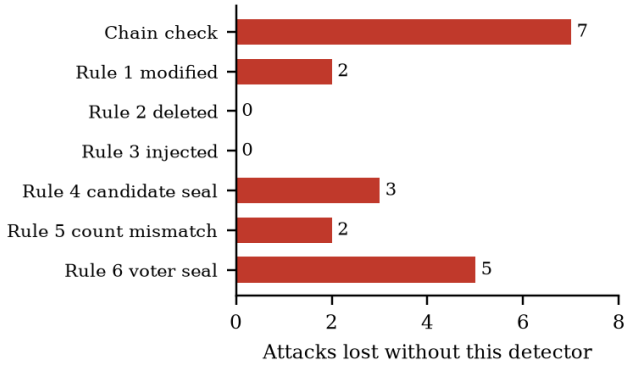


Fig. 3. Attacks lost when one detector is removed from the final design.

C. Accuracy and correctness

Across 399 sweeps over 120 randomised clean elections (bulk registration, late voters, ballots, closing) there were 0 false alarms. When 1, 5 or 20 ballots were modified, deleted or injected, the sweep named exactly those ballots: precision and recall were 1.0 in all nine cases. The tally agreed with an independent implementation that explores every tie-break in 500 of 500 elections of 3 to 8 candidates and 5 to 300 ballots (66 had a tie path).

D. Passwords

Moving from unsalted SHA-256 to salted PBKDF2 makes each offline guess about 361,996 times slower and prevents precomputed tables. A sign-in takes a median 496 ms of hashing; the round count is configurable for slower hosts.

E. Performance

TABLE III. Median time in ms with the keyed file ledger.

Ballots	Sweep	Chain	Tally	Cast p50
100	6	2	6	14
1,000	22	13	14	45
10,000	269	121	95	251
100,000	5,985	2,731	2,689	4,658

Audit and tally grow roughly linearly (Fig. 4). At 10,000 ballots, typical of a large campus election, the sweep takes 269 ms; at 100,000 it takes 6.0 s. With the file ledger the cost of casting grows with the ledger because each append rewrites the file; against MongoDB Atlas over a mobile hotspot an append took about 140 ms regardless of length. With 1 to 16 simultaneous voters there were no errors and the chain and sweep were intact after every run (up to 16 casts per second on a laptop).

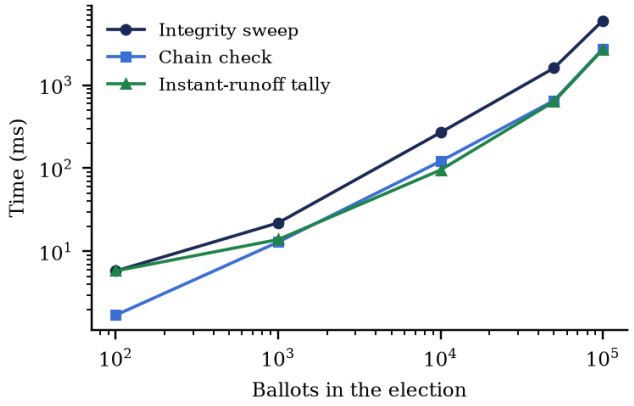


Fig. 4. Audit time against election size (log scales).

F. Abuse and privacy probes

All 13 abuse attempts through the public API were refused, including twenty simultaneous casts by one voter, of which exactly one was stored; six of seven attempts to break the rules from a database console were rejected by the engine, the exception being a dangling candidate reference because SQLite enforces foreign keys per connection (the sweep reports it). The ballot and voter tables share no column other than the election; an ordering attack on 150 casts recovered no voter. An attacker with two database copies and one cast in between links a voter to a ballot with probability one over the number of casts, a known limit of any design that stores a voted flag.

VII. COMPARISON WITH RELATED SYSTEMS

TABLE IV. Documentation survey of 33 systems (the rest: not documented or absent).

Capability (documented by the system)	Yes	Partial
F1 Ranked choice with round-by-round transcript	2	11
F3 Integrity rules inside the relational database	1	3
F5 Keyed tamper-evident ledger, reconciling sweep, published attacks	0	16
F6 Emailed one-time code at sign-in	2	13
F8 Voter receipt	9	3

Of 33 systems whose own pages we read, none documents all of F1, F3, F5 and F6, and none documents F5 in the strict

sense used here. "Not documented" means we did not find it, not that it is absent. This is a statement about documentation, not about the products.

Where related systems are stronger is the substance. Helios, Belenios and ElectionGuard let a voter check that the recorded ballot is the one cast and let anyone check the tally; PolyVote's receipts show only that a ballot is recorded and counted. A hostile administrator who holds the ledger key defeats PolyVote and does not defeat them. Those systems have years of use and public analysis; PolyVote has none. Its claim is cost and fit: free, open source, one command to run, and a detection result that anyone can re-run.

VIII. LIMITATIONS AND THREATS TO VALIDITY

The 31 attacks were written by the authors, so the in-scope result shows coverage of the attacks we thought of, not of all attacks; an independently written list is future work. Competing designs are placed in measured classes from documentation, not run. An insider with the ledger key and both stores can forge a consistent chain; publishing the chain head to an external witness [14] would address this. There is no account lock-out, no session expiry, the login reveals whether a registration number exists, and API responses carry no security headers. Receipts do not prove content [25]. The file ledger is slow at 100,000 ballots. Timings come from one laptop. One early prevention test was flaky and was fixed; we disclose it. No user study has been run, so we make no usability claim.

IX. REPRODUCIBILITY AND FUTURE WORK

Every number in this paper is produced by scripts in the repository: `run_all.py` runs the 31 scenarios in both key modes, the ablation and the design profiles; `extras.py` runs the accuracy, correctness, password, performance and concurrency measurements; `static_metrics.py` and `shots.py` record code, dependency and interface measurements. A full run takes under ten minutes on a laptop and writes JSON and CSV files from which the tables and figures are drawn. The attack scenarios operate on a throwaway copy of the application, so a reader can add their own without touching a real election.

Four extensions would most strengthen the evidence. First, an attack list written by people who did not build the system. Second, an external witness that receives the chain head, which would extend detection to a key holder [14]. Third, account lock-out, session expiry and response headers, which the probes show are missing. Fourth, a task-based usability study with voters and administrators; we have prepared a protocol, consent form, task cards and a standard System Usability Scale analysis, but no participant has yet been tested.

X. CONCLUSION

A keyed receipt chain, a reconciling sweep and a keyed roster seal let a small organisation detect silent edits by a database administrator at modest cost. No single control sufficed: the combination detected all 28 in-scope attacks, the plain chain with a receipt sweep 11, and a conventional application 11, with no false alarms on clean elections. We do not offer end-to-end verifiability and we do not claim that the attack list is complete. Code, attack scripts, raw results, 21 figures and a 32-page dossier are at https://github.com/Tharakeshtj/polyvote-election-platform; a demo runs at https://polyvote-bacse202.onrender.com.

REFERENCES

[1] B. Adida, “Helios: Web-based Open-Audit Voting,” in Proc. 17th USENIX Security Symposium (USENIX Security 08), San Jose, CA, 2008. [Online]. Available: https://www.usenix.org/conference/17th-usenix-security-symposium/helios-web-based-open-audit-voting

[2] V. Cortier, P. Gaudry, and S. Glondu, “Belenios: A Simple Private and Verifiable Electronic Voting System,” in Foundations of Security, Protocols, and Equational Reasoning (Lecture Notes in Computer Science), Springer, 2019, pp. 214–238. doi: 10.1007/978-3-030-19052-1_14.

[3] M. R. Clarkson, S. Chong, and A. C. Myers, “Civitas: Toward a Secure Voting System,” in Proc. 2008 IEEE Symposium on Security and Privacy, 2008, pp. 354–368. doi: 10.1109/sp.2008.32.

[4] P. Y. A. Ryan, D. Bismark, J. Heather, S. Schneider, and Z. Xia, “Prêt à Voter: a Voter-Verifiable Voting System,” IEEE Transactions on Information Forensics and Security, vol. 4, no. 4, pp. 662–673, 2009. doi: 10.1109/TIFS.2009.2033233.

[5] D. Chaum et al., “Scantegrity II: End-to-End Verifiability by Voters of Optical Scan Elections Through Confirmation Codes,” IEEE Transactions on Information Forensics and Security, vol. 4, no. 4, pp. 611–627, 2009. doi: 10.1109/tifs.2009.2034919.

[6] D. Springall et al., “Security Analysis of the Estonian Internet Voting System,” in Proc. 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS '14), 2014, pp. 703–715. doi: 10.1145/2660267.2660315.

[7] M. A. Specter, J. Koppel, and D. Weitzner, “The Ballot is Busted Before the Blockchain: A Security Analysis of Voatz, the First Internet Voting Application Used in U.S. Federal Elections,” in Proc. 29th USENIX Security Symposium (USENIX Security 20), 2020, pp. 1535–1553. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/specter

[8] M. A. Specter and J. A. Halderman, “Security Analysis of the Democracy Live Online Voting System,” in Proc. 30th USENIX Security Symposium (USENIX Security 21), 2021, pp. 3077–3092. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/specter-security

[9] National Academies of Sciences, Engineering, and Medicine, Securing the Vote: Protecting American Democracy. National Academies Press, Washington, DC, 2018. doi: 10.17226/25120.

[10] S. Park, M. Specter, N. Narula, and R. L. Rivest, “Going from bad to worse: from Internet voting to blockchain voting,” Journal of Cybersecurity, vol. 7, no. 1, 2021. doi: 10.1093/cybsec/tyaa025.

[11] S. Haber and W. S. Stornetta, “How to time-stamp a digital document,” Journal of Cryptology, vol. 3, no. 2, pp. 99–111, 1991. doi: 10.1007/BF00196791.

[12] B. Schneier and J. Kelsey, “Secure audit logs to support computer forensics,” ACM Transactions on Information and System Security, vol. 2, no. 2, pp. 159–176, 1999. doi: 10.1145/317087.317089.

[13] S. A. Crosby and D. S. Wallach, “Efficient Data Structures for Tamper-Evident Logging,” in Proc. 18th USENIX Security Symposium (USENIX Security 09), Montreal, Canada, 2009. [Online]. Available: https://www.usenix.org/conference/usenixsecurity09/technical-sessions/presentation/efficient-data-structures-tamper-evident

[14] B. Laurie, A. Langley, and E. Kasper, “Certificate Transparency,” IETF, RFC 6962, 2013 (Experimental). [Online]. Available: https://www.rfc-editor.org/info/rfc6962

[15] R. T. Snodgrass, S. S. Yao, and C. Collberg, “Tamper Detection in Audit Logs,” in Proc. 2004 VLDB Conference, 2004, pp. 504–515. doi: 10.1016/b978-012088469-8.50046-2.

[16] K. E. Pavlou and R. T. Snodgrass, “Forensic analysis of database tampering,” ACM Transactions on Database Systems, vol. 33, no. 4, pp. 1–47, 2008. doi: 10.1145/1412331.1412342.

[17] P. Antonopoulos et al., “SQL Ledger: Cryptographically Verifiable Data in Azure SQL Database,” in Proc. 2021 International Conference on Management of Data (SIGMOD '21), 2021, pp. 2437–2449. doi: 10.1145/3448016.3457558.

[18] T. R. Magrino, R. L. Rivest, E. Shen, and D. Wagner, “Computing the Margin of Victory in IRV Elections,” in Proc. 2011 Electronic Voting Technology

Workshop / Workshop on Trustworthy Elections (EVT/WOTE '11), San Francisco, CA, 2011. [Online]. Available: https://www.usenix.org/conference/evtwote-11/computing-margin-victory-irv-elections

[19] M. Blom, P. J. Stuckey, and V. Teague, "RAIRE: Risk-Limiting Audits for IRV Elections," arXiv preprint arXiv:1903.08804 [cs.DS], 2019. [Online]. Available: https://arxiv.org/abs/1903.08804

[20] S. Aulbach, T. Grust, D. Jacobs, A. Kemper, and J. Rittinger, "Multi-tenant databases for software as a service: schema-mapping techniques," in Proc. 2008 ACM SIGMOD International Conference on Management of Data, 2008, pp. 1195–1206. doi: 10.1145/1376616.1376736.

[21] M. Sönmez Turan, E. Barker, W. Burr, and L. Chen, "Recommendation for Password-Based Key Derivation: Part 1: Storage Applications," NIST Special Publication 800-132, National Institute of Standards and Technology, 2010. doi: 10.6028/NIST.SP.800-132.

[22] OWASP, "Password Storage Cheat Sheet," OWASP Cheat Sheet Series, OWASP Foundation. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (accessed Oct. 22, 2026).

[23] D. Temoshok et al., "Digital Identity Guidelines: Authentication and Authenticator Management," NIST Special Publication 800-63B-4, National Institute of Standards and Technology, 2025. doi: 10.6028/NIST.SP.800-63b-4.

[24] S. L. Garfinkel, "Email-based identification and authentication: an alternative to PKI?," IEEE Security & Privacy, vol. 1, no. 6, pp. 20–26, 2003. doi: 10.1109/MSECP.2003.1253564.

[25] D. Chaum, "Secret-ballot receipts: true voter-verifiable elections," IEEE Security & Privacy, vol. 2, no. 1, pp. 38–47, 2004. doi: 10.1109/MSECP.2004.1264852.